```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
```

```
nn_architecture = [
    {"input_dim": 2, "output_dim": 4, "activation": "relu"},
    {"input_dim": 4, "output_dim": 6, "activation": "relu"},
    {"input_dim": 6, "output_dim": 6, "activation": "relu"},
    {"input_dim": 6, "output_dim": 4, "activation": "relu"},
    {"input_dim": 4, "output_dim": 1, "activation": "sigmoid"},
]
```

The `nn_architecture` dictionary defines the structure of a neural network. Each dictionary within the list represents a single layer in the network:

- **`input_dim`**: This specifies the number of input features or neurons from the previous layer that feed into the current layer. For the first layer, it's the number of features in your dataset (in this case, 2 for the 2D moon dataset).
- **`output_dim`**: This specifies the number of neurons in the current layer, which also becomes the `input_dim` for the next layer. This value determines the dimensionality of the output of the current layer.
- **`activation`**: This specifies the activation function to be used for the neurons in the current layer. Common activation functions include 'relu' (Rectified Linear Unit) and 'sigmoid' (Sigmoid function), which introduce non-linearity into the network, allowing it to learn complex patterns.

```
def init_layers(nn_architecture, seed=99):
    np.random.seed(seed)
    params_values = {}  # A dictionary is used to store parameters (weights and biases) for each layer.
                        # It allows accessing parameters by descriptive string keys like 'W1', 'b1'.
    for idx, layer in enumerate(nn_architecture):
        layer_idx = idx + 1
        # 'str(layer_idx)' converts the integer layer index to a string.
        # This is necessary to concatenate it with 'W' to form unique string keys like 'W1', 'W2' for the dictionary.
        params_values["W" + str(layer_idx)] = np.random.randn(
            layer["output_dim"], layer["input_dim"]) * 0.1
        # Similarly, 'str(layer_idx)' is used here for bias keys like 'b1', 'b2'.
        params_values["b" + str(layer_idx)] = np.zeros((layer["output_dim"], 1))
    return params_values
```

```
init_layers(nn_architecture)
```

```
{'W1': array([[-0.01423588,  0.20572217],
        [ 0.02832619,  0.1329812 ],
        [-0.01546219, -0.00690309],
        [ 0.07551805,  0.08256466]]),
 'b1': array([[0.],
        [0.],
        [0.]]),
 'W2': array([[-0.01130692, -0.23678376, -0.01670494,  0.0685398 ],
        [ 0.00235001,  0.04562013,  0.02704928, -0.14350081],
        [ 0.08828171, -0.05800817, -0.05015653,  0.05909533],
        [-0.07316163,  0.02617555, -0.08557956, -0.01875259],
        [-0.03734863, -0.0461971 , -0.08164661, -0.00451233],
        [ 0.01213278,  0.09259528, -0.05738197,  0.00527031]]),
 'b2': array([[0.],
        [0.],
        [0.],
        [0.],
        [0.]]),
 'W3': array([[ 0.22073106,  0.03918219,  0.04827134,  0.0433334 , -0.17042917,
         -0.02439081],
        [-0.21397038,  0.08613227,  0.17002844, -0.05287848,  0.17634779,
         -0.11216078],
        [-0.11919342,  0.05527319, -0.08159809, -0.04966468,  0.10862256,
         -0.09746753],
        [-0.02821358, -0.01172141,  0.03785473,  0.07321946, -0.0103571 ,
         -0.11987063],
        [ 0.10100356,  0.28753603,  0.08203126,  0.05606115, -0.03756422,
         -0.02521043],
        [-0.13896134,  0.06173323, -0.0135787 ,  0.1287905 , -0.10369944,
          0.13643321]]),
 'b3': array([[0.],
        [0.],
        [0.],
        [0.],
        [0.]]),
 'W4': array([[-0.03099566, -0.06111171, -0.04831058, -0.06089837, -0.20883353,
           0.0639322 ],
        [ 0.0774304 ,  0.12785694,  0.0705276 ,  0.06559774, -0.1678502 ,
          0.01831099],
        [-0.11332241, -0.02790857,  0.13966199,  0.00322194, -0.26136608,
         -0.10015776],
        [-0.0567511 , -0.0225658 ,  0.09380238,  0.08367841,  0.08121485,
```

```
         0.0232307 ]]),
 'b4': array([[0.],
       [0.],
       [0.],
       [0.]]),
 'W5': array([[-0.02951077, -0.0361676 ,  0.04321151,  0.09339585]]),
 'b5': array([[0.]])}
```

```
def sigmoid(x):
  return 1 / (1 + np.exp(-x))

def relu(x):
    return np.maximum(0, x)

def sigmoid_backword(dA, x):
  sig = sigmoid(x)
  return dA * sig * (1 - sig)

def relu_backword(dA, x):
  dx = np.array(dA, copy=True)
  dx[x <= 0] = 0
  return dx
```

```
def convert_prob_intro_class(probs):
  """Converts probabilties into binary classes (0 or 1)."""
  return (probs >= 0.5).astype(int)
```

```
def single_layer_forward_propagation(A_prev, W_curr, b_curr, activation="relu"):
    Z_curr = np.dot(W_curr, A_prev) + b_curr
    if activation == "relu":
        activation_func = relu
    elif activation == "sigmoid":
        activation_func = sigmoid
    else:
      raise Exception('Non-supported activation function')
    return activation_func(Z_curr), Z_curr
```

```
single_layer_forward_propagation(-0.2, 0.03, 4)
```

```
(np.float64(3.994), np.float64(3.994))
```

```
def full_forward_propagation(X, params_values, nn_architecture):
    memory = {}
    A_curr = X
    for idx, layer in enumerate(nn_architecture):
        layer_idx = idx + 1
        A_prev = A_curr
        A_curr, Z_curr = single_layer_forward_propagation(
            A_prev, params_values["W" + str(layer_idx)],
            params_values["b" + str(layer_idx)],
            layer["activation"])
        memory["A" + str(idx)] = A_prev
        memory["Z" + str(layer_idx)] = Z_curr
    return A_curr, memory
```

```
# Cross Entropy
def get_cost_value(Y_hat, Y):
    m = Y_hat.shape[1]
    cost = -1 / m * (np.dot(Y, np.log(Y_hat).T) + np.dot(1 - Y, np.log(1 - Y_hat).T))
    return np.squeeze(cost)
```

```
def get_accuracy_value(Y_hat, Y):
    Y_hat_class = convert_prob_intro_class(Y_hat)
    return (Y_hat_class == Y).mean()
```

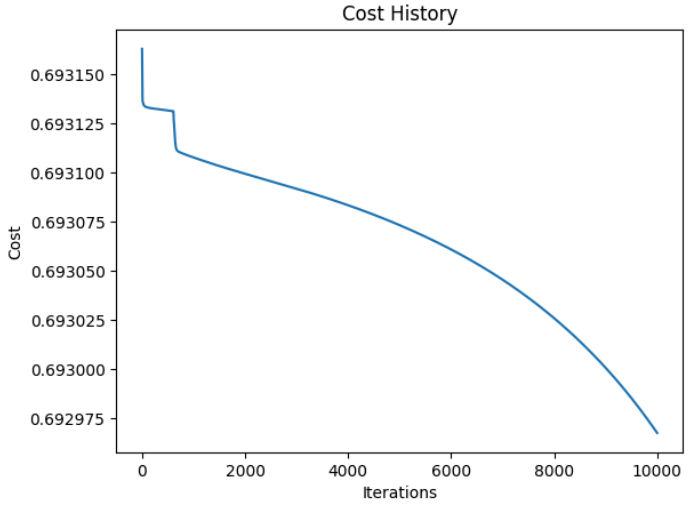
```
def single_layer_backrward_propagation(dA_curr, W_curr, b_curr, Z_curr, A_prev, activation="relu"):
  m = A_prev.shape[1]
  if activation == "relu":
    backward_activation_func = relu_backword
  elif activation == "sigmoid":
    backward_activation_func = sigmoid_backword
  else:
    raise Exception('Non-supported activation function')

  dZ_curr = backward_activation_func(dA_curr, Z_curr)
  dW_curr = np.dot(dZ_curr, A_prev.T) / m
  db_curr = np.sum(dZ_curr, axis=1, keepdims=True) / m
  dA_prev = np.dot(W_curr.T, dZ_curr)
  return dA_prev, dW_curr, db_curr
```

```
def full_backward_propagation(Y_hat, Y, memory, params_values, nn_architecture):
  grads_values = {}
  Y = Y.reshape(Y_hat.shape)
  dA_prev = - (np.divide(Y, Y_hat) - np.divide(1 - Y, 1 - Y_hat))
```

```
  for layer_idx_prev, layer in reversed(list(enumerate(nn_architecture))):
    layer_idx_curr = layer_idx_prev + 1
    dA_prev, dW_curr, db_curr = single_layer_backrward_propagation(
          dA_prev,
          params_values["W" + str(layer_idx_curr)],
          params_values["b" + str(layer_idx_curr)],
          memory["Z" + str(layer_idx_curr)],
          memory["A" + str(layer_idx_prev)],
          activation=layer["activation"]
        )
    grads_values["dW" + str(layer_idx_curr)] = dW_curr
    grads_values["db" + str(layer_idx_curr)] = db_curr
  return grads_values
```

```
def update(params_values, grads_values, nn_architecture, learning_rate):
  for layer_idx, layer in enumerate(nn_architecture):
    layer_idx = layer_idx + 1
    params_values["W" + str(layer_idx)] -= learning_rate * grads_values["dW" + str(layer_idx)]
    params_values["b" + str(layer_idx)] -= learning_rate * grads_values["db" + str(layer_idx)]
  return params_values

def train(X, Y, nn_architecture, epochs, learning_rate):
  params_values = init_layers(nn_architecture, 2)
  cost_history = []
  accuracy_history = []

  for i in range(epochs):
    Y_hat, memory = full_forward_propagation(X, params_values, nn_architecture)
    cost = get_cost_value(Y_hat, Y)
    cost_history.append(cost)
    accuracy = get_accuracy_value(Y_hat, Y)
    accuracy_history.append(accuracy)

    grads_values = full_backward_propagation(Y_hat, Y, memory, params_values, nn_architecture)
    params_values = update(params_values, grads_values, nn_architecture, learning_rate)

    if i % 1000 == 0:
      print(f"Iteration: {i} - cost: {cost:.5f} - accuracy: {accuracy:.5f}")
  return params_values, cost_history, accuracy_history

x , y = make_moons(n_samples=1000, noise=0.1, random_state=42)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

#Transpose for the NN(Features x samples)
x_train = x_train.T
y_train = y_train.reshape(1 , -1)

#train
params_values, cost_history, accuracy_history = train(x_train, y_train, nn_architecture, epochs=10000, learning_rate=0.01)

plt.plot(cost_history)
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.title("Cost History")
plt.show()
```

```
Iteration: 0 - cost: 0.69316 - accuracy: 0.50000
Iteration: 1000 - cost: 0.69311 - accuracy: 0.79125
Iteration: 2000 - cost: 0.69310 - accuracy: 0.86125
Iteration: 3000 - cost: 0.69309 - accuracy: 0.85750
Iteration: 4000 - cost: 0.69308 - accuracy: 0.85500
Iteration: 5000 - cost: 0.69307 - accuracy: 0.85250
Iteration: 6000 - cost: 0.69306 - accuracy: 0.85125
Iteration: 7000 - cost: 0.69305 - accuracy: 0.84625
Iteration: 8000 - cost: 0.69303 - accuracy: 0.84000
Iteration: 9000 - cost: 0.69300 - accuracy: 0.83500
```



Start coding or generate with AI.